AWS Master Notes: Cloud Practitioner + DevOps Zero to Hero

Scope

These notes combine AWS Cloud Practitioner fundamentals with a practical AWS DevOps learning path: cloud basics, identity, compute, storage, networking, databases, monitoring, billing, security, integrations, CI/CD, containers, infrastructure as code, migration, and interview preparation.

Included Sources

These notes now incorporate:

- the original AWS learning playlist used for the first draft
- the additional playlist provided by the user: https://www.youtube.com/playlist?list=PL6XT0grm_TfgtwtwUit305qS-HhDvb4du
- selected official AWS documentation, especially Amazon EBS and Amazon Data Lifecycle Manager docs, to strengthen the operational details with primary-source guidance

Learning Map

1. What is Cloud Computing
2. Identity and Access Management
3. Budget
4. EC2
5. Security Groups
6. Storage
7. Amazon Machine Image
8. Scalability and High Availability
9. Elastic Load Balancing
10. Auto Scaling Group
11. Amazon S3
12. Databases

13. Other Compute Services
14. Deployments
15. AWS Global Infrastructure
16. Cloud Integrations
17. Cloud Monitoring
18. VPC
19. Security and Compliance
20. Machine Learning
21. Account Management and Billing
22. Advanced Identity
23. Other Services
24. AWS Architecting and Ecosystem
25. Preparing for the Cloud Practitioner Exam

1. What Is Cloud Computing

Cloud computing means renting IT resources on demand instead of buying, hosting, and maintaining everything yourself.

Key ideas:

- On-demand self-service: resources can be provisioned without waiting for manual infrastructure setup.
- Broad network access: services are reachable over the internet or private network links.
- Resource pooling: AWS serves many customers from shared physical infrastructure.
- Rapid elasticity: capacity can scale up or down quickly.
- Measured service: you pay for actual usage.

Why organizations move to cloud:

- lower upfront capital expense
- faster experimentation
- global reach
- easier scaling
- reduced undifferentiated heavy lifting
- better resilience options

Core cloud service models:

- IaaS: infrastructure such as virtual machines, storage, and networking
- PaaS: managed platforms where you focus more on code than infrastructure

- SaaS: fully managed software delivered to end users

Cloud deployment models:

- Public cloud: shared provider infrastructure
- Private cloud: dedicated environment for one organization
- Hybrid cloud: mix of on-premises and cloud
- Multi-cloud: use of more than one cloud provider

Main benefits of AWS:

- trade fixed cost for variable cost
- stop guessing capacity
- achieve economies of scale
- increase speed and agility
- deploy globally in minutes

Important mindset:

- design for failure
- automate repetitive work
- build elastic systems
- use managed services when possible

2. Identity and Access Management

IAM controls who can do what in AWS.

Core identities:

- Root user: the account owner with full access; use only for critical account-level tasks
- IAM user: long-term identity for a specific person or system
- IAM group: collection of users with shared permissions
- IAM role: temporary assumable identity, commonly used by AWS services or cross-account access

Permission building blocks:

- Policy: JSON document defining allowed or denied actions
- Effect: Allow or Deny
- Action: service operation such as ‘s3:GetObject‘
- Resource: target AWS resource
- Condition: optional logic such as IP, MFA, or tag-based restrictions

Best practices:

- enforce least privilege

- enable MFA, especially for privileged users
- avoid using the root user for daily work
- prefer roles over long-lived access keys
- rotate credentials and review permissions regularly
- give people separate accounts or users, not shared credentials

How access usually happens:

- Console access: password plus MFA
- CLI or SDK access: access key and secret key, ideally through temporary credentials
- Service-to-service access: IAM roles

Useful IAM tools:

- IAM Access Analyzer: helps find overly broad or external access
- IAM credential report: shows account credential posture
- IAM Access Advisor: shows what services a principal actually used

High-yield comparison:

- IAM user: best for a human or legacy workload needing long-lived credentials
- IAM role: best for temporary access, services, applications, and cross-account usage

3. Budget

AWS cost control starts with visibility, alerts, and disciplined service choices.

Main cost tools:

- AWS Budgets: create thresholds and alerts
- Cost Explorer: analyze spend over time
- Cost and Usage Report: detailed billing data for reporting
- Billing dashboard: invoices, total charges, tax, and payment details

Best practices for budget control:

- set monthly budgets early
- create alerts before reaching the full limit
- tag resources for team, project, and environment
- stop unused resources
- right-size compute and storage
- prefer managed services only when they still fit the cost model

Common hidden cost sources:

- idle EC2 instances

- unattached EBS volumes
- NAT Gateway data processing
- public data transfer out
- snapshots retained forever
- over-provisioned databases

Important pricing concepts:

- pay-as-you-go
- reserved commitment discounts
- Savings Plans
- free tier
- data transfer charges

4. EC2

EC2 is AWS virtual server infrastructure.

An EC2 decision usually includes:

- operating system
- instance family and size
- CPU and memory needs
- storage type
- security group
- network placement
- purchasing option

Instance families:

- General purpose: balanced compute, memory, and networking
- Compute optimized: high CPU workloads
- Memory optimized: in-memory databases, caching, analytics
- Storage optimized: high IOPS or high-throughput local storage
- Accelerated computing: GPU, ML, inference, graphics

EC2 storage choices:

- EBS: persistent block storage
- Instance store: temporary local storage that disappears if the instance stops or terminates
- EFS: shared file storage for Linux workloads

Bootstrap concept:

- User data runs commands at first boot to install packages, configure the

server, or start applications

Purchasing options:

- On-Demand: flexible, no commitment
- Reserved Instances: discount for long-term commitment
- Savings Plans: flexible commitment-based discount
- Spot Instances: cheap unused capacity, but can be interrupted
- Dedicated Hosts or Instances: isolation-focused options

Best practices:

- choose the smallest viable instance and scale later
- separate application state from compute when possible
- use roles instead of embedding credentials
- place instances behind load balancers for production

5. Security Groups

Security Groups are stateful virtual firewalls attached to ENIs and commonly associated with EC2 instances.

Important properties:

- they allow traffic; they do not contain explicit deny rules
- they are stateful, so return traffic is automatically allowed
- they control inbound and outbound rules
- they can reference CIDR blocks or other security groups
- they operate at the instance or network interface level

Default behavior:

- inbound denied by default
- outbound allowed by default

Good patterns:

- separate SSH administration rules from application rules
- allow only required ports
- restrict management access to known IPs
- reference application security groups instead of wide IP ranges when possible

Troubleshooting clue:

- timeout usually points to network filtering or routing
- connection refused usually points to the service not listening correctly

6. Storage

AWS storage can be grouped into object, block, file, and archival storage.

Block storage:

- EBS: persistent block storage for EC2
- supports snapshots
- volume types differ by performance profile

File storage:

- EFS: shared, elastic NFS file system for Linux workloads
- FSx: managed file systems for specific ecosystems such as Windows, Lustre, NetApp, and OpenZFS

Object storage:

- S3: scalable object storage for backups, static websites, logs, data lakes, and archives

Local ephemeral storage:

- instance store provides high-speed local storage, but data is lost when the instance lifecycle ends

How to think about the choices:

- use block storage when the OS or application needs a disk
- use file storage when multiple instances need the same file system
- use object storage for unstructured data and web-scale durability

7. Amazon Machine Image

An AMI is a machine template used to launch EC2 instances.

An AMI can contain:

- operating system
- application packages
- configuration
- launch permissions
- mapped storage settings

Why AMIs matter:

- standardize builds
- reduce manual setup
- speed up provisioning
- support golden image strategy

Common AMI sources:

- AWS-provided AMIs
- marketplace AMIs
- custom AMIs created from existing instances

Important notes:

- AMIs are region-specific
- you can copy AMIs between regions
- custom AMIs help enforce consistent configuration across environments

8. Scalability and High Availability

Scalability is the ability to handle growth. High availability is the ability to stay up despite failures.

Two scaling approaches:

- vertical scaling: make one machine bigger
- horizontal scaling: add more machines or instances

High availability patterns:

- use multiple Availability Zones
- use load balancing
- distribute state into resilient managed services
- avoid single points of failure

Exam-important idea:

- horizontal scaling is usually preferred in cloud-native design because it improves both elasticity and resilience

Design principles:

- keep compute stateless
- externalize sessions when needed
- decouple tiers
- automate replacement instead of manual repair

9. Elastic Load Balancing

Elastic Load Balancing distributes incoming traffic across healthy targets.

Benefits:

- single entry point for clients
- automatic health checks

- better availability across AZs
- support for SSL termination
- easier horizontal scaling

Main load balancer types:

- ALB: layer 7, HTTP and HTTPS aware, path- and host-based routing
- NLB: layer 4, very high performance, TCP and UDP use cases
- GWLB: designed for third-party virtual appliances such as firewalls

Good fit:

- use ALB for web apps and microservices
- use NLB for high-throughput or low-latency network traffic
- use GWLB when traffic must pass through managed security appliances

Related concepts:

- target groups
- listener rules
- health checks
- sticky sessions
- SSL termination

10. Auto Scaling Group

An Auto Scaling Group maintains the right number of EC2 instances.

Core settings:

- desired capacity
- minimum capacity
- maximum capacity

What ASG gives you:

- automatic replacement of unhealthy instances
- scheduled or dynamic scaling
- better availability across AZs
- cost control when demand falls

Scaling triggers can be based on:

- CPU utilization
- request count
- custom CloudWatch metrics
- schedules

Important supporting object:

- Launch template defines AMI, instance type, security groups, storage, and user data

Best practice:

- combine ASG with ELB and health checks for resilient applications

11. Amazon S3

S3 is object storage built for durability, scalability, and broad use cases.

S3 basics:

- data is stored as objects in buckets
- objects have key names and metadata
- buckets are region-scoped and globally unique by name

Common uses:

- backups
- static website hosting
- logs
- media storage
- data lakes
- software artifact storage

Important features:

- versioning
- lifecycle policies
- replication
- event notifications
- encryption
- bucket policies
- access control through IAM and resource policies

Major storage classes:

- Standard: frequent access
- Intelligent-Tiering: automatic optimization when access patterns change
- Standard-IA: infrequent access
- One Zone-IA: cheaper infrequent access with less resilience scope
- Glacier Instant Retrieval
- Glacier Flexible Retrieval
- Glacier Deep Archive

Key security points:

- block public access by default unless there is a real business need
- encrypt data at rest and in transit
- use bucket policies carefully

12. Databases

AWS offers purpose-built databases instead of forcing one engine for every workload.

Relational options:

- RDS: managed relational databases
- Aurora: cloud-optimized relational engine with MySQL or PostgreSQL compatibility

NoSQL options:

- DynamoDB: key-value and document store, highly scalable and serverless

Caching:

- ElastiCache: Redis or Memcached for low-latency access

Analytics and specialized engines:

- Redshift: data warehouse
- Neptune: graph database
- DocumentDB: document database
- Keyspaces: Cassandra-compatible
- Timestream: time-series workloads

Selection guide:

- choose RDS or Aurora for structured relational data and joins
- choose DynamoDB for massive scale, low-latency key-value patterns
- choose ElastiCache when performance bottlenecks come from repeated reads

High-yield comparison:

- RDS: relational, structured, SQL, typically fixed schema
- DynamoDB: NoSQL, key-value or document, massive scale, low operational effort
- Aurora: relational like RDS but more cloud-optimized and higher performance

13. Other Compute Services

AWS compute is broader than EC2.

Important services:

- Lambda: serverless function execution
- ECS: managed container orchestration for Docker workloads
- EKS: managed Kubernetes control plane
- Fargate: serverless runtime for containers
- Elastic Beanstalk: platform-oriented application deployment
- App Runner: simple app deployment for web applications and APIs
- Batch: managed batch job scheduling and execution
- Lightsail: simplified VPS-style service

Quick selection:

- choose Lambda for event-driven short-running code
- choose ECS or EKS for containerized applications
- choose Fargate when you want containers without managing servers
- choose Beanstalk or App Runner for simpler application deployment

14. Deployments

Deployment strategy matters as much as infrastructure.

Key deployment services:

- CloudFormation: infrastructure as code
- CodeBuild, CodeDeploy, CodePipeline: CI/CD services
- Systems Manager: automation and patching support

Common deployment patterns:

- all-at-once
- rolling update
- blue/green
- canary
- immutable deployment

What to optimize:

- low downtime
- rollback safety
- automation
- repeatability
- auditability

Important principle:

- infrastructure and deployment logic should be versioned, repeatable, and preferably automated

15. AWS Global Infrastructure

AWS global design affects latency, resilience, and compliance.

Main building blocks:

- Region: geographic area with multiple data centers
- Availability Zone: isolated data center grouping within a region
- Edge location: used by services like CloudFront for low-latency delivery
- Regional Edge Cache: intermediate cache layer
- Local Zone: extends selected services closer to users
- Wavelength Zone: ultra-low-latency workloads near telecom networks

Exam-important service scope:

- some services are global, such as IAM
- many services are regional, such as EC2 and RDS
- resources may also be zonal, such as an individual EC2 instance in an AZ

Design takeaway:

- multiple AZs improve resilience
- multiple regions improve disaster recovery and global reach

16. Cloud Integrations

Modern AWS systems communicate through messaging, events, and APIs.

Main integration services:

- SQS: durable message queues for decoupling
- SNS: pub/sub notifications and fan-out
- EventBridge: event routing across services and custom applications
- API Gateway: managed API front door
- Step Functions: workflow orchestration
- MQ: managed ActiveMQ and RabbitMQ

High-yield comparison:

- SQS: pull-based queue, one consumer or worker pattern
- SNS: push-based pub/sub to many subscribers
- EventBridge: event bus with routing rules and SaaS or AWS integrations

Architecture lesson:

- decoupled systems fail less catastrophically and scale more predictably

17. Cloud Monitoring

Observability means metrics, logs, events, traces, configuration history, and audit history.

Key services:

- CloudWatch: metrics, alarms, logs, dashboards, events
- CloudTrail: API and account activity history
- AWS Config: resource configuration history and compliance checking
- X-Ray: request tracing
- Trusted Advisor: recommendations on cost, security, performance, and resilience

Common uses:

- alarm on CPU or error rate
- centralize logs
- audit who changed what
- detect configuration drift

High-yield comparison:

- CloudWatch answers "How is the system behaving?"
- CloudTrail answers "Who did what in the account?"
- AWS Config answers "What changed in resource configuration?"

18. VPC

The VPC is the private networking layer for AWS resources.

Core building blocks:

- CIDR block
- subnets
- route tables
- Internet Gateway
- NAT Gateway
- security groups
- network ACLs

Public versus private subnet:

- public subnet has a route to an Internet Gateway
- private subnet does not directly expose resources to the internet

Security Group versus NACL:

- Security Group: stateful, attached to instances or ENIs, allow rules
- NACL: stateless, subnet-level, allow and deny rules

Other networking concepts:

VPC peering:

- VPC peering creates a private network connection between two VPCs so resources in one VPC can talk to resources in the other using private IP addresses.
- It works for VPCs in the same account, different AWS accounts, and even different regions.
- Traffic stays on the AWS backbone instead of moving over the public internet.
- VPC peering is non-transitive. If VPC A is peered with VPC B, and VPC B is peered with VPC C, VPC A cannot automatically reach VPC C through VPC B.
- It is best for smaller and simpler architectures where only a limited number of VPCs need direct connectivity.
- As environments grow, peering relationships can become difficult to manage because each connection is one-to-one and route tables must be maintained carefully.

Transit Gateway:

- Transit Gateway acts like a central networking hub that connects many VPCs and on-premises networks through a single routing layer.
- Instead of creating many separate peering relationships, each VPC or VPN can attach to the Transit Gateway.
- It simplifies large-scale network design because routing is centralized rather than managed in a mesh of many connections.
- It supports transitive routing, which means connected networks can communicate through the hub when routes are configured.
- Transit Gateway is a strong fit for enterprises with multiple AWS accounts, multiple VPCs, branch offices, and hybrid cloud requirements.
- The main idea is operational simplicity at scale, especially when compared with managing large numbers of VPC peering links.

VPC endpoints:

- VPC endpoints let resources inside a VPC access supported AWS services privately without sending traffic through the public internet, an Internet Gateway, a NAT Gateway, or a VPN connection.
- This improves both security and network design by keeping traffic inside the AWS private network.
- There are two commonly discussed categories:
- Gateway endpoints, mainly for Amazon S3 and DynamoDB.

- Interface endpoints, powered by PrivateLink, for many AWS services and supported third-party services.
- Gateway endpoints are added to route tables, while interface endpoints create elastic network interfaces inside subnets.
- VPC endpoints are useful when private workloads need to reach services like S3, Systems Manager, Secrets Manager, or ECR without public exposure.
- They also help reduce dependence on NAT Gateway traffic for some service access patterns.

Site-to-Site VPN:

- Site-to-Site VPN creates an encrypted tunnel between an on-premises network and AWS, usually terminating at a Virtual Private Gateway or Transit Gateway.
- It is commonly used to extend a corporate network into AWS so on-premises systems can privately access cloud resources.
- Traffic is encrypted over the internet, which makes it more secure than plain public connectivity.
- It is faster to set up than dedicated private connectivity, so it is useful for hybrid cloud adoption, branch connectivity, disaster recovery, and temporary network extensions.
- Compared with Direct Connect, VPN is usually quicker and cheaper to start with, but latency and throughput are less predictable because it still uses internet paths.
- Many organizations use VPN first, then add Direct Connect later for more stable enterprise connectivity.

Direct Connect:

- Direct Connect provides a dedicated private network connection from an organization’s data center or colocation environment to AWS.
- Unlike Site-to-Site VPN, it does not depend on the public internet for the main transport path.
- It is used when teams need more consistent latency, higher bandwidth, stronger hybrid connectivity, or lower large-scale data transfer costs.
- Direct Connect is often chosen for enterprise integrations, large migrations, backup traffic, and regulated environments that need predictable connectivity.
- It can be combined with VPN for encrypted failover or layered resilience.
- The key trade-off is that Direct Connect usually takes longer to provision and involves more planning than VPN.

Bastion host:

- A bastion host is a hardened server placed in a public subnet and used as a controlled jump point to reach instances in private subnets.

- Administrators first connect to the bastion host, then connect onward to private instances that are not directly exposed to the internet.
- This pattern reduces the number of systems with public access and centralizes administrative entry points.
- A bastion host should be tightly locked down with restricted IP access, strong authentication, patching, logging, and minimal installed software.
- In many modern AWS environments, Session Manager is often preferred over bastion hosts because it avoids opening SSH to the internet and provides better centralized access control and auditability.
- Bastion hosts are still important to understand because they remain common in traditional or mixed administration models.

Best practice:

- keep databases in private subnets
- use least-exposed networking
- document CIDR planning carefully before environments grow

19. Security and Compliance

Security in AWS is built from both AWS-managed controls and customer-managed controls.

Shared responsibility model:

- AWS secures the cloud
- customers secure what they run in the cloud

Key security services:

- KMS: key management
- Secrets Manager: secrets storage and rotation
- Systems Manager Parameter Store: parameter and secret storage
- WAF: web application filtering
- Shield: DDoS protection
- GuardDuty: threat detection
- Inspector: vulnerability management
- Macie: sensitive data discovery
- Security Hub: security posture aggregation
- Detective: investigation support
- Artifact: compliance reports and documents

Security habits:

- enable MFA

- rotate or eliminate long-lived secrets
- encrypt data at rest and in transit
- log all important activity
- apply least privilege
- isolate environments
- continuously review access and compliance posture

20. Machine Learning

Cloud Practitioner-level ML knowledge is service awareness, not deep model math.

Important services:

- SageMaker: build, train, and deploy models
- Rekognition: image and video analysis
- Comprehend: NLP and sentiment or entity detection
- Lex: chatbots
- Polly: text-to-speech
- Transcribe: speech-to-text
- Translate: machine translation
- Textract: extract text and structure from documents
- Forecast and Personalize: specialized prediction and recommendation use cases

How to think about them:

- use managed AI services when the use case is well defined
- use SageMaker when the organization needs end-to-end custom model workflows

21. Account Management and Billing

Account structure affects governance and cost clarity.

Important topics:

- AWS Organizations
- consolidated billing
- multi-account strategy
- cost allocation tags
- support plans
- invoice and payment management

Why multi-account matters:

- separate dev, test, and prod

- isolate business units
- control blast radius
- simplify billing and governance

Cost optimization levers:

- right-size resources
- choose suitable purchasing models
- delete idle assets
- monitor data transfer
- review storage class choices

22. Advanced Identity

Advanced identity focuses on centralized access and federation.

Key services and concepts:

- IAM Identity Center for workforce access across accounts
- STS for temporary credentials
- AssumeRole for cross-account access
- federation with external identity providers
- permission boundaries
- service control policies in Organizations

Important distinctions:

- IAM handles identities and permissions within an account context
- IAM Identity Center is stronger for centralized workforce access across many accounts
- STS provides temporary credentials

Best practice:

- prefer short-lived credentials and federated access wherever practical

23. Other Services

Cloud Practitioner study also benefits from broad awareness of major adjacent services.

Examples:

- Route 53: DNS and routing policies
- CloudFront: CDN and edge caching
- SES: email sending

- Athena: serverless SQL queries on S3
- Glue: ETL and data catalog
- Kinesis: streaming data platform
- OpenSearch Service: managed search and analytics
- AppSync: GraphQL APIs
- QuickSight: BI dashboards

Goal at this level:

- know what category of problem each service solves
- do not try to memorize every feature

24. AWS Architecting and Ecosystem

Architecture in AWS is about selecting the right managed components, minimizing operational risk, and aligning with best practices.

AWS Well-Architected pillars:

- Operational Excellence
- Security
- Reliability
- Performance Efficiency
- Cost Optimization
- Sustainability

High-level design habits:

- design for elasticity
- automate infrastructure
- build decoupled components
- prefer managed services
- monitor everything important
- plan for failure

Ecosystem awareness:

- AWS Marketplace for third-party software
- partner network for consulting and solutions
- AWS documentation, training, and certification paths
- support plans for operational assistance

25. Preparing for the Cloud Practitioner Exam

Study strategy:

- master service categories before memorizing details
- focus on use cases and trade-offs
- learn global versus regional versus zonal scope
- understand pricing at a high level
- learn the security shared responsibility boundary

High-yield comparisons to memorize:

- Security Group versus NACL
- S3 versus EBS versus EFS
- RDS versus DynamoDB
- ALB versus NLB versus GWLB
- CloudWatch versus CloudTrail versus Config
- IAM user versus role
- Reserved Instances versus Savings Plans versus Spot
- SNS versus SQS versus EventBridge

Question-solving technique:

- eliminate answers that require too much manual effort when AWS has a managed service
- if the prompt stresses elastic, resilient, highly available, or serverless, lean toward managed AWS-native designs
- if the prompt stresses cheapest short-term compute, think Spot
- if the prompt stresses audit history, think CloudTrail
- if the prompt stresses configuration compliance, think AWS Config
- if the prompt stresses metrics and alarms, think CloudWatch

Final revision checklist:

- know the major storage, compute, networking, database, security, and monitoring services
- understand basic billing controls and optimization
- know identity best practices
- know what multi-AZ and multi-region mean
- know the purpose of core integration services

Quick Comparison Sheet

Storage Comparison

- S3: object storage for files, logs, backups, analytics, and archival workflows
- EBS: persistent block storage for EC2

- EFS: shared file storage for Linux systems
- Instance store: very fast temporary local storage

Network Security Comparison

- Security Group: stateful, instance-level, allow-only
- NACL: stateless, subnet-level, allow-and-deny
- WAF: layer 7 application protection
- Shield: DDoS protection

Load Balancer Comparison

- ALB: HTTP or HTTPS web routing
- NLB: TCP or UDP high-performance traffic
- GWLB: traffic steering for virtual appliances

Identity Comparison

- IAM user: long-lived identity
- IAM role: temporary assumable identity
- IAM Identity Center: centralized workforce access

Monitoring Comparison

- CloudWatch: metrics, alarms, logs
- CloudTrail: account and API audit history
- AWS Config: resource configuration and compliance tracking

Practical Learning Advice

- Build a free-tier-safe lab with IAM users, budgets, one EC2 instance, one S3 bucket, and CloudWatch alarms.
- Practice reading AWS console pages with service scope in mind: account-level, region-level, or AZ-level.
- Draw simple architectures while studying. Visualizing traffic flow and service relationships improves retention.
- Learn services by problem category, not by memorizing alphabetical lists.
- When two services look similar, ask three questions: What data shape is involved? How is scale handled? Who manages operations?

DevOps Expansion Path

This part extends the AWS foundation into hands-on DevOps workflows. The emphasis shifts from service awareness to building, automating, securing, deploying, and

operating workloads on AWS.

DevOps Roadmap Overview

A practical AWS DevOps learning path often follows this order:

1. set up the AWS account and basic access model
2. launch EC2 and practice SSH access
3. design VPC networking and secure connectivity
4. manage DNS and public endpoints with Route 53
5. automate infrastructure with AWS CLI and CloudFormation
6. build CI/CD with CodeCommit, CodePipeline, CodeBuild, and CodeDeploy
7. add monitoring, alarms, and event-driven automation
8. move into containers with ECR, ECS, and EKS
9. strengthen secrets management, auditing, migration, and production best practices

AWS Account Setup and Console Readiness

Before building anything serious, the AWS account needs a safe baseline.

Checklist:

- protect the root account with MFA
- create an IAM admin user or federated admin access path
- define billing alerts and budgets
- choose a default region intentionally
- create separate environments for dev, test, and prod where possible
- start with naming and tagging standards early

Good tags usually include:

- ‘Project‘
- ‘Environment‘
- ‘Owner‘
- ‘CostCenter‘
- ‘ManagedBy‘

This pays off later in automation, billing analysis, and operations.

Hands-On EC2 for DevOps

In a DevOps path, EC2 is not just a compute service. It becomes the first place to practice deployment, access control, service exposure, and troubleshooting.

Important skills:

- launch an instance with the correct AMI and size
- attach a security group with only required ports
- use key pairs correctly
- connect with SSH
- install software through user data or configuration scripts
- host a simple application and validate external connectivity

Useful debugging flow for an unreachable app:

1. check instance state
2. verify security group inbound rules
3. verify subnet routing and internet gateway if public
4. confirm the app is listening on the expected port
5. test locally on the instance with ‘curl‘ or ‘ss -tulpn‘
6. inspect OS firewall settings when relevant

Route 53 and DNS Operations

Route 53 connects friendly domain names to AWS resources.

Main records to know:

- ‘A‘: IPv4 address mapping
- ‘AAAA‘: IPv6 mapping
- ‘CNAME‘: alias to another name
- ‘ALIAS‘: AWS-aware mapping to services like ALB and CloudFront
- ‘MX‘: mail routing
- ‘TXT‘: verification and policy records

Important routing policies:

- simple routing
- weighted routing
- latency-based routing
- failover routing
- geolocation routing
- multi-value answer routing

Operational use cases:

- point a domain to an ALB
- direct traffic to CloudFront
- perform health-based failover
- split traffic across blue/green environments

Secure VPC Project Thinking

A secure VPC lab is one of the best AWS exercises because it combines networking, compute, routing, and identity.

A strong mental model looks like this:

- public subnet: bastion hosts, public ALBs, or internet-facing components
- private subnet: application servers, internal services, databases
- Internet Gateway: inbound and outbound internet path for public resources
- NAT Gateway: outbound internet access for private resources
- route tables: define traffic direction
- security groups: instance-level stateful controls
- NACLs: subnet-level stateless controls

Best practices in a secure VPC design:

- expose only the minimum public surface area
- keep databases private
- use separate subnets across AZs
- avoid broad ‘0.0.0.0/0‘ rules for admin ports
- use bastion hosts, SSM Session Manager, or private access patterns for administration

AWS CLI for Automation

The AWS CLI is the bridge between manual learning and automation.

Why it matters:

- repeat tasks faster than the console
- script provisioning and operations
- understand service APIs more clearly
- integrate AWS operations into CI/CD pipelines

Core commands to get comfortable with:

- ‘aws configure‘
- ‘aws sts get-caller-identity‘
- ‘aws ec2 describe-instances‘
- ‘aws s3 ls‘
- ‘aws cloudformation describe-stacks‘
- ‘aws iam list-users‘

Best practices:

- verify the current identity before destructive operations

- use profiles for multiple accounts
- prefer IAM roles or SSO-backed access rather than static keys
- script idempotent operations where possible

CloudFormation and Infrastructure as Code

CloudFormation turns infrastructure into version-controlled templates.

Core concepts:

- template: YAML or JSON definition of resources
- stack: a deployed instance of the template
- parameters: runtime inputs
- outputs: values exported after deployment
- change set: preview of infrastructure changes
- stack policy: protection against unsafe updates

Why IaC matters:

- repeatable infrastructure
- better review process
- faster environment creation
- easier rollback and auditability

CloudFormation habits to build:

- keep templates modular
- use parameters for environment differences
- store templates in version control
- review change sets before applying critical updates
- use outputs to connect stacks cleanly

CodeCommit

CodeCommit is AWS-managed Git hosting.

What to know:

- supports Git workflows similar to GitHub or GitLab
- integrates well with AWS IAM and CodePipeline
- useful in environments that want tighter AWS-native control

Operational considerations:

- design branch permissions carefully
- use IAM policies for repository access
- integrate with pipeline triggers and code review practices

Even if teams prefer GitHub in practice, understanding CodeCommit helps in AWS-native CI/CD design.

CodePipeline

CodePipeline orchestrates software delivery stages.

Typical stages:

- Source
- Build
- Test
- Approval
- Deploy

What it provides:

- end-to-end delivery visibility
- stage-based automation
- integration with CodeBuild, CodeDeploy, ECS, Lambda, and third-party sources

Strong use cases:

- application delivery pipelines
- infrastructure deployment pipelines
- cross-account deployment patterns

Best practice:

- keep each stage single-purpose
- fail early on validation and tests
- separate build artifacts from deployment logic

CodeBuild

CodeBuild is managed build execution.

Main building blocks:

- build project
- build environment
- ‘buildspec.yml‘
- artifacts
- logs and reports

What ‘buildspec.yml‘ commonly contains:

- install phase
- pre_build phase

- build phase
- post_build phase
- artifact configuration

Typical DevOps use cases:

- unit tests
- linting
- Docker image builds
- packaging deployment bundles
- security scanning

Best practices:

- keep build environments minimal and reproducible
- store secrets outside the buildspec body
- cache dependencies where it meaningfully reduces build time

CodeDeploy

CodeDeploy automates deployments to compute targets.

Supported targets include:

- EC2 or on-prem servers
- Lambda
- ECS

Important concepts:

- application
- deployment group
- deployment configuration
- appspec file
- rollback behavior

Why CodeDeploy matters:

- reduces manual deployment risk
- supports blue/green strategies
- allows controlled rollout and rollback

Blue/green idea:

- blue is the current live environment
- green is the new environment
- traffic shifts after validation
- rollback becomes easier because the old environment remains available

temporarily

CloudWatch Operations

CloudWatch is more than metrics. In DevOps work it becomes the operational feedback layer.

You should know how to use:

- metrics
- alarms
- dashboards
- log groups
- log streams
- metric filters
- notifications through SNS

Common operational patterns:

- alarm when CPU or memory behavior crosses a threshold
- alarm when application error logs spike
- create dashboards for service health
- send notifications to email, chatops, or incident systems

Good alarm design:

- avoid noise from overly sensitive thresholds
- distinguish warning from critical conditions
- tie alerts to actionability

Lambda for Event-Driven Automation

Lambda runs code without managing servers.

Great use cases:

- event-driven glue code
- log processing
- automation tasks
- scheduled jobs
- API backends
- reacting to S3, EventBridge, or SNS events

Important limits and design rules:

- execution time is limited
- functions should stay stateless

- package size and dependencies matter
- IAM permissions should be tightly scoped

DevOps examples:

- trigger remediation from an alarm
- resize images on S3 upload
- rotate or validate resources on a schedule

EventBridge and CloudWatch Events

EventBridge routes events between AWS services and custom applications.

Why it matters:

- supports event-driven architecture
- reduces tight coupling
- enables automation across services

Typical event-driven flow:

1. a service emits an event
2. EventBridge matches the rule
3. the target receives it, such as Lambda, Step Functions, SNS, or SQS

Common use cases:

- respond to EC2 state changes
- enforce tagging or compliance automatically
- trigger workflows from deployment events

CloudFront and Edge Delivery

CloudFront is AWS CDN and edge caching service.

Benefits:

- lower latency for global users
- caching to reduce origin load
- TLS termination and domain support
- integration with S3, ALB, and custom origins

Important concepts:

- distribution
- origin
- cache behavior
- TTL
- invalidation

- edge locations

Common patterns:

- serve static content from S3 through CloudFront
- front an ALB or API for global acceleration
- pair with WAF for edge security

ECR and Container Image Management

ECR stores container images close to AWS deployment targets.

Core tasks:

- create repositories
- authenticate Docker to ECR
- tag images correctly
- push and pull images
- manage image lifecycle

Best practices:

- use immutable version tags where possible
- scan images for vulnerabilities
- delete stale tags through lifecycle policies
- separate dev and prod image flows clearly

ECS and Container Operations

ECS runs containers without requiring a full Kubernetes control-plane model.

Important objects:

- cluster
- task definition
- task
- service
- capacity provider

Deployment choices:

- ECS on EC2
- ECS on Fargate

When ECS is a strong fit:

- teams want AWS-native container orchestration
- workloads are containerized but do not need Kubernetes complexity
- operational simplicity matters more than portability

EKS and Kubernetes on AWS

EKS provides managed Kubernetes control plane capabilities.

What to understand:

- EKS manages the Kubernetes control plane
- worker nodes still need lifecycle and networking decisions
- VPC networking and IAM integration are important
- Kubernetes manifests or Helm manage workloads

When EKS is a strong fit:

- the team already knows Kubernetes
- portability and ecosystem tooling matter
- workloads need richer orchestration patterns than simpler platforms provide

Trade-off:

- EKS is powerful but more operationally demanding than ECS or App Runner

Systems Manager and Secrets Manager

Operational maturity improves a lot when administration and secrets handling are centralized.

Systems Manager capabilities:

- Session Manager for shell access without opening SSH to the internet
- Run Command for remote execution
- Patch Manager
- Automation documents
- Parameter Store

Secrets Manager capabilities:

- secure secret storage
- rotation workflows
- application integration
- auditing of secret access patterns

Best practice:

- avoid placing secrets in code, AMIs, or shell history
- prefer Session Manager over public SSH exposure when possible

Terraform on AWS

Terraform is multi-cloud infrastructure as code and is widely used in DevOps teams.

Core Terraform concepts:

- provider
- resource
- variable
- output
- state
- module
- plan
- apply

Why Terraform matters in AWS environments:

- consistent environment creation
- reusable infrastructure modules
- better collaboration through version control
- strong support across AWS services

State management best practice:

- store state remotely
- protect it with locking where possible
- treat state as sensitive operational data

A common Terraform AWS project includes:

- VPC
- subnets
- route tables
- security groups
- EC2 instances
- load balancer

CloudTrail and AWS Config Deep Dive

These two services become critical once an environment grows beyond simple labs.

CloudTrail is for:

- API audit history
- security investigations
- governance and accountability

AWS Config is for:

- resource inventory
- configuration history

- compliance rules
- drift awareness

Examples:

- CloudTrail tells you who deleted or changed a resource
- Config tells you how a resource configuration changed over time

Elastic Load Balancer Deep Dive

Beyond the basic overview, load balancers are essential in production delivery.

Things to practice:

- create target groups
- register or deregister targets
- configure health checks correctly
- observe failed target behavior
- attach SSL certificates through ACM where needed

Why this matters operationally:

- bad health check paths can make healthy apps look down
- weak target design can create partial outages
- load balancers are central to blue/green and rolling deployments

AWS Migration Strategies and Tools

Cloud migration is not just lift-and-shift. It is about choosing the right strategy for each workload.

Common migration patterns:

- rehost: move with minimal changes
- replatform: small optimizations during the move
- refactor: redesign for cloud-native architecture
- repurchase: move to a SaaS replacement
- retire: remove what is no longer needed
- retain: keep some systems where they are for now

Things to assess before migration:

- application dependencies
- data gravity
- downtime tolerance
- compliance needs
- networking design

- cost after migration

Best Practices and Job Preparation

Strong AWS DevOps preparation usually centers on patterns, not memorizing every feature.

High-value habits:

- think in managed services first
- isolate environments clearly
- automate infrastructure and deployments
- monitor for both health and security
- use least privilege and short-lived credentials
- prefer repeatable pipelines over manual fixes

Good interview themes:

- why choose ECS vs EKS
- how to secure access to EC2
- how CloudWatch, CloudTrail, and Config differ
- how blue/green deployment works
- how to design a secure VPC
- how to store and rotate secrets safely

Real-World Project Pattern with RDS

A realistic AWS application project often combines:

- Route 53 for DNS
- CloudFront or ALB for traffic entry
- application compute on EC2, ECS, or EKS
- RDS for relational persistence
- CloudWatch for visibility
- Secrets Manager for database credentials
- CI/CD for deployment automation

Important design choices around RDS:

- private subnet placement
- security group restriction to application tiers only
- automated backups
- Multi-AZ for production resilience
- parameter tuning and instance sizing

Added Revision Sheet for AWS DevOps

- AWS CLI is the first step from manual work to automation.
- CloudFormation and Terraform both solve infrastructure as code, but Terraform is more cross-platform.
- CodePipeline orchestrates delivery; CodeBuild builds; CodeDeploy deploys.
- Lambda and EventBridge are central to event-driven automation.
- ECR stores images; ECS and EKS run containers.
- Systems Manager improves remote administration; Secrets Manager improves secret handling.
- CloudTrail is for auditing; AWS Config is for configuration compliance.
- CloudFront improves content delivery and edge performance.
- Route 53 handles DNS and advanced traffic policies.
- RDS should usually live in private subnets with tightly controlled access.

Detailed Deep-Dive Companion

This companion section expands the major subtopics into longer revision notes. The goal is not just to memorize service names, but to understand why each service exists, how to choose between similar options, and what design trade-offs show up in real AWS environments.

Cloud Foundations Deep Dive

On-demand self-service means teams do not need to raise an infrastructure ticket every time they need a new server, storage volume, or managed service. In cloud environments, developers and operators can provision resources through the console, CLI, SDK, or infrastructure-as-code templates. This reduces waiting time and increases delivery speed, but it also makes governance more important because fast provisioning without controls can lead to security or cost problems.

Broad network access means services are reachable over standard networks and protocols. In practice, that includes web console access, API access, CLI usage, and application-to-application communication. The cloud is powerful because users, systems, and automation can interact with services from many places, but that also means identity, network segmentation, encryption, and monitoring become critical.

Resource pooling refers to the provider sharing underlying infrastructure across many customers while isolating each customer logically. Customers do not manage racks, hardware procurement, or facility operations. Instead, they consume slices of compute, storage, and networking on demand. This drives economies of scale and lets cloud providers offer better resilience and global reach than many small organizations could build on their own.

Rapid elasticity means resources can scale with demand. The key lesson is that cloud design should assume demand changes over time. Good architecture uses load balancers, stateless services, managed storage, and autoscaling where possible so the system can grow without a complete redesign.

Measured service means billing is tied to usage. This creates flexibility but also requires operational discipline. When teams forget to stop instances, clean up snapshots, or delete unused load balancers, costs continue to accumulate. Cloud literacy therefore includes cost awareness, not just technical deployment skill.

IAM Deep Dive

The root user is the highest-privilege identity in an AWS account. It has full authority over billing, account settings, and many recovery paths. Because it is extremely powerful, it should be protected with MFA and used rarely. Daily work should happen through IAM or federated identities.

IAM users are long-lived identities with credentials such as passwords or access keys. They are simple to understand, but in mature environments the preferred pattern is often to reduce reliance on static access keys. Long-lived credentials are risky because they can leak into shell history, repositories, CI systems, or screenshots.

IAM groups simplify user management by attaching policies at the group level rather than to each user individually. Groups help keep permission assignment consistent, especially when multiple users need the same access profile.

IAM roles are central to AWS design. A role has permissions but is assumed temporarily rather than logged into directly like a traditional user. Services such as EC2, Lambda, ECS, and CodeBuild commonly assume roles. Cross-account access also depends heavily on roles. Roles reduce the risk of credential sprawl because credentials are temporary and can be rotated automatically by AWS.

Policies define authorization logic. When reading a policy, focus on four things: who receives it, which actions are allowed or denied, which resources are targeted, and whether any conditions apply. Conditions are especially important because they let teams enforce safer rules, such as requiring MFA for sensitive actions or restricting access to specific source IPs or tags.

Least privilege is the most important IAM mindset. Start with the smallest permission set needed, observe what the workload actually uses, and refine access over time. Overly broad access might make a demo easier, but in production it increases blast radius when credentials are misused or stolen.

Budget, Billing, and Pricing Deep Dive

AWS Budgets is an alerting tool, not just a dashboard. It is most useful when configured early, before teams begin experimenting freely. Good budget design includes separate thresholds, such as 50 percent, 80 percent, and 100 percent of expected spend, so corrective action can happen before the invoice becomes painful.

Cost Explorer is useful for trend analysis. It helps answer questions such as which service category is growing, which tagged environment costs the most, or which purchase option changes might reduce spend. It is especially helpful after a workload has been running for some time and real usage patterns emerge.

The Cost and Usage Report is the most detailed billing dataset. Organizations often use it when they need reporting pipelines, allocation across teams, or deep analysis in Athena, QuickSight, or third-party cost platforms.

On-Demand pricing is best when workloads are unpredictable, temporary, or early in their lifecycle. Reserved pricing models and Savings Plans become attractive when usage is steady enough to justify commitment. Spot Instances are excellent for interruption-tolerant workloads such as batch processing, distributed build jobs, or certain analytics tasks, but they are a poor fit for systems that cannot tolerate sudden termination.

Data transfer is one of the most overlooked billing areas. New learners focus on compute size, but networking decisions such as heavy internet egress, NAT Gateway usage, inter-region transfer, or poor CDN strategy can create surprisingly large charges. Cost optimization therefore includes architecture choices, not just instance rightsizing.

EC2 Deep Dive

EC2 is flexible because it exposes core infrastructure decisions directly: operating system, compute shape, storage, network placement, identity attachment, and bootstrapping. This flexibility is why it remains foundational, but it also means there is more to configure correctly compared with higher-level platforms.

Instance families exist because not all workloads are the same. Web servers and small business apps often fit general-purpose instances. Build systems, encoding jobs, and CPU-heavy analytics may need compute-optimized instances. In-memory caching, high-volume relational databases, and memory-intensive applications lean toward memory-optimized families. When evaluating any workload, ask whether it is limited by CPU, memory, storage throughput, or network throughput.

User data is powerful because it turns first boot into an automation step. Teams often use it to install packages, pull configuration, register services, or start applications. However, user data should be used thoughtfully. Large complex provisioning logic can become hard to debug, so many teams eventually combine user

data with configuration tools, golden images, or containerized startup flows.

EBS is the default persistent disk choice for EC2. It survives instance stops and can be snapshotted, resized, encrypted, and moved across lifecycle operations. Instance store, by contrast, is temporary. It can be extremely fast, but it should only be used for data that can be rebuilt or lost without serious impact.

When choosing purchasing models, treat cost and reliability together. Spot is attractive, but only when the application design can tolerate interruption. Savings Plans and Reserved options help reduce cost for steady workloads, but only if usage is predictable enough to justify commitment.

Security Groups and Network Controls Deep Dive

Security Groups are stateful and attached at the instance or ENI level. Their mental model is "allow what is needed, and deny everything else by omission." Because they are stateful, return traffic is automatically permitted. This makes them easier to reason about than NACLs in many day-to-day cases.

Network ACLs operate at subnet level and are stateless. That means return traffic must be explicitly allowed. NACLs are useful when subnet-wide controls are needed, but they are also easier to misconfigure because both directions matter explicitly. In many environments, Security Groups do most of the real workload protection, while NACLs add a coarse-grained boundary.

When troubleshooting connectivity, think through the full path: DNS resolution, route table, subnet type, internet or NAT access, Security Group rules, NACL rules, service listener state, and local OS firewall settings. Many AWS issues are not caused by a single component but by the interaction of several layers.

Storage Deep Dive

EBS is block storage, which means the operating system sees it like a disk. That makes it ideal for boot volumes, databases running on EC2, or applications expecting a traditional filesystem on attached storage. Volume type matters because not every disk needs the same performance. Some workloads need high IOPS, while others mainly need capacity and moderate throughput.

EFS is shared file storage for Linux-style workloads. Multiple instances can mount the same file system, which makes it useful for shared content, common assets, or legacy applications that expect a POSIX-compatible shared file system. EFS trades some simplicity and elasticity benefits for cost and performance characteristics that differ from block storage.

FSx exists because some workloads need specialized file protocols or performance

profiles. It is less common at beginner level, but it becomes important when teams need Windows file shares, high-performance compute file systems, or NetApp-style enterprise storage features.

S3 is not a filesystem and not a block disk. It is object storage. That means data is stored as objects identified by keys in buckets. This design allows massive scale, high durability, and flexible lifecycle management, but it also means applications must interact with objects differently than they would with mounted file systems.

EBS Snapshot, Restore, and Backup Automation Deep Dive

The additional playlist brings more day-to-day EC2 storage operations into focus, especially snapshot creation, restore flows, copy strategy, and lifecycle automation. According to AWS documentation, EBS snapshots are point-in-time backups stored in Amazon S3 that you cannot access directly through the normal S3 console or S3 APIs. The first snapshot of a volume is full, and later snapshots are incremental, so storage cost depends on the data blocks written rather than the raw provisioned disk size.

Snapshot creation is asynchronous. AWS creates the snapshot request immediately, but the snapshot stays in the pending state while blocks are transferred to S3. The volume can continue serving traffic during that time, but the snapshot includes only the data written when the request was made, not data still sitting in application or OS cache. That is why AWS recommends pausing writes or unmounting the filesystem when consistency matters, and stopping an instance before taking a root volume snapshot when you need the safest result.

Restoring from a snapshot means creating a new EBS volume from it. That new volume begins as an exact replica, but the snapshot blocks are initialized in the background, so the volume can experience higher latency until initialization finishes. AWS documentation now describes two important performance options here: Fast Snapshot Restore for fully initialized volumes at creation time, and a provisioned volume initialization rate when teams want more predictable restore timing. If you choose a provisioned initialization rate, AWS uses that path instead of Fast Snapshot Restore.

Amazon Data Lifecycle Manager is the main automation service for this topic. AWS docs say DLM can automate the creation, retention, and deletion of EBS snapshots and EBS-backed AMIs, but it cannot manage snapshots or AMIs created by other means, and it cannot automate instance store-backed AMIs. In practice, that means teams should standardize backup creation through DLM if they want predictable retention behavior rather than mixing several unrelated manual processes.

DLM policies can target either individual volumes or all volumes attached to tagged

instances. That distinction matters. Volume-targeted policies are useful when a specific disk needs its own schedule. Instance-targeted policies are better when an application spans multiple attached volumes and should be backed up as one coordinated unit. AWS also notes that target tags are case sensitive and that newly attached volumes on a targeted instance are included in later policy runs, so tag hygiene becomes part of backup reliability.

For application-consistent snapshots, AWS documents the use of DLM with Systems Manager pre and post scripts. This is especially relevant for Windows Volume Shadow Copy Service workloads and for databases that need I/O flush and freeze behavior before a snapshot starts. The operational lesson is important: automated snapshots are not automatically application-consistent snapshots. A recoverable backup strategy depends on how the workload behaves before and after the snapshot request.

Copying snapshots is another operational layer. AWS docs note that snapshot copies in the same account and Region using the same KMS key are always incremental. Across Regions or accounts, copies can also be incremental if a previous copy still exists in the destination, has not been archived, and uses compatible encryption; otherwise AWS performs a full copy. User-defined tags are not copied automatically, copied snapshots do not automatically keep Fast Snapshot Restore enabled, and encrypted shared snapshots require access to the customer-managed KMS key as well as the snapshot itself. These details matter in disaster recovery planning because storage cost, restore speed, and permissions all change based on the copy path.

Archive tier is designed for long-term retention rather than quick restore. AWS states that archived snapshots are full snapshots, require a minimum archive period of 90 days, cannot be used to create volumes until they are restored, and do not support Fast Snapshot Restore while archived. That makes archive useful for compliance or infrequent recovery, but not for rapid operational rollback.

The same lifecycle mindset also applies to AMIs. DLM can automate EBS-backed AMI lifecycles, which is useful for golden image refresh pipelines and standard server baselines. In real environments, teams usually use snapshots for data protection, AMIs for server standardization, and a combination of both when they want predictable rebuilds of both application hosts and their attached storage.

For interviews and production design, the key takeaway is that EBS backup strategy is a design problem, not only a console feature. Teams need to decide backup frequency, crash consistency versus application consistency, cross-Region or cross-account copy needs, expected restore time, archive retention, encryption ownership, and the tag model that drives automation. That is the level where EC2 storage administration turns into real AWS operations knowledge.

S3 Deep Dive

Versioning protects against accidental deletion and overwrite. Once enabled, an object can have multiple versions, which is extremely useful for rollback, auditability, and operational recovery. Lifecycle policies then help control cost by transitioning objects between storage classes or expiring stale versions and incomplete uploads.

Replication helps with resilience and compliance. Cross-Region Replication can support disaster recovery, global presence, or regulatory requirements. Same-Region Replication can support operational separation and additional internal patterns.

Bucket policies are resource-based policies, while IAM policies are identity-based. Understanding this difference matters because access decisions often depend on both. Public access control on S3 can become tricky if bucket policies, ACLs, object ownership settings, and block public access controls are not aligned correctly.

Storage classes should be selected according to access pattern, durability expectations, and retrieval urgency. Standard is easiest for active content. Intelligent-Tiering helps when access patterns are uncertain. Glacier options reduce cost for colder data, but retrieval behavior and latency differ significantly.

Database Deep Dive

RDS is managed relational infrastructure. It reduces operational effort around backups, patching, failover, and basic administration, but you still choose instance type, storage, engine, and performance settings. Aurora goes further with a cloud-optimized relational architecture and is often chosen for performance and scaling advantages over standard RDS engines.

DynamoDB is purpose-built for high-scale, low-latency NoSQL access patterns. Its design favors predictable key-based access rather than relational joins and ad hoc relational querying. The most important DynamoDB learning is data modeling around access patterns rather than normalizing tables the way you would in a relational database.

ElastiCache improves performance by placing frequently accessed data in memory. Many applications use Redis for session storage, caching, queues, rate limiting, or leaderboards. The main lesson is that caching improves speed but introduces cache invalidation and consistency design questions.

Redshift is optimized for analytical queries rather than transactional application traffic. Neptune serves graph relationships. Timestream supports time-series data. The broad exam takeaway is that AWS expects teams to choose purpose-built data services rather than forcing every workload into a single engine.

Load Balancing and High Availability Deep Dive

An Application Load Balancer is ideal when routing decisions depend on HTTP details such as hostnames, paths, or headers. It understands web traffic and is often used for APIs, microservices, and websites. A Network Load Balancer is stronger when the requirement is raw TCP or UDP performance, low latency, or preserving source details at layer 4. Gateway Load Balancer is more specialized and is mainly relevant for traffic steering through security appliances.

Health checks are critical. A load balancer is only as smart as the health signal it receives. If the health endpoint is too weak, the load balancer may send traffic to a partially broken instance. If it is too strict, healthy targets may be removed unnecessarily. Designing health endpoints is therefore part of application reliability, not just infrastructure configuration.

Auto Scaling Groups improve resilience because unhealthy instances can be replaced automatically. Combined with multiple AZ placement and load balancing, ASGs support both availability and elasticity. The deeper lesson is that cloud-native systems should be ready for instance replacement at any time rather than assuming a server will live forever.

VPC Deep Dive

CIDR planning matters early. Poor CIDR design creates pain later when teams need peering, hybrid connectivity, or environment expansion. Overlapping ranges are especially problematic because they block straightforward routing between networks.

Subnets are not just a way to separate IP ranges. They are a control boundary for routing, network ACL application, and public versus private exposure. Public resources belong in subnets with an Internet Gateway route. Private resources belong in subnets that rely on NAT or private connectivity patterns.

NAT Gateway allows private resources to reach outbound internet destinations without becoming directly reachable from the internet. It is operationally simple but can become a notable cost line item. That means architects should understand when NAT is actually needed and when VPC endpoints or other private access models can reduce dependence on it.

VPC peering is best when a small number of VPCs need direct private communication. It is simple and fast, but because it is non-transitive, it becomes harder to manage as network count increases. Transit Gateway solves that scaling problem by centralizing connectivity.

VPC endpoints are one of the best private-networking patterns to learn well. They reduce public exposure and can improve security posture significantly. For example,

private EC2 instances can pull from S3 or call Secrets Manager without needing internet access at all.

Site-to-Site VPN is excellent for fast hybrid connectivity, testing, branch extension, and disaster recovery. Direct Connect is stronger for stable enterprise-grade connectivity with predictable performance. Bastion hosts are a traditional administrative pattern, but modern environments often shift toward Systems Manager Session Manager because it improves auditability and reduces exposed management ports.

Monitoring, Logging, and Audit Deep Dive

CloudWatch is the primary operational monitoring surface. Metrics show numerical health, logs show text output and error details, dashboards provide visibility, and alarms create action. Good monitoring begins with meaningful metrics and actionable alerts rather than alerting on everything.

CloudTrail records API activity in the account. This is vital for forensic analysis, governance, and security reviews. If someone asks who changed a security group, deleted a role, or modified a bucket policy, CloudTrail is one of the first services to check.

AWS Config tracks resource configuration state over time and can evaluate resources against compliance rules. If CloudTrail answers who performed an action, Config helps answer how the configuration changed and whether it still meets policy.

Trusted Advisor gives recommendations, but teams should treat it as a guide rather than a full operational strategy. Human review and service-specific monitoring remain essential.

Security Services Deep Dive

KMS is foundational because encryption strategy often depends on key management decisions. Teams must understand whether AWS-managed keys are enough or whether customer-managed keys with tighter control, rotation policy, and access governance are required.

Secrets Manager is ideal when applications need managed secret rotation or structured secret storage. Parameter Store is simpler and often sufficient for many configuration values and some secrets, especially when advanced rotation workflows are not needed. The main lesson is to keep secrets out of code, AMIs, and plain-text operational scripts.

WAF protects applications at layer 7, especially web traffic patterns such as common exploit attempts, abusive requests, and some bot or filtering use cases. Shield

focuses on DDoS protection. GuardDuty, Inspector, Macie, Security Hub, and Detective each contribute to visibility and posture but solve different parts of the security lifecycle.

The shared responsibility model must be understood precisely. AWS secures the underlying cloud infrastructure, but customers still control identity, network policy, application security, data protection configuration, and the way workloads are deployed.

DevOps Toolchain Deep Dive

Route 53 is not just DNS. In real environments it becomes part of resilience, cutover, migration, and deployment strategy. Weighted routing can help controlled rollout. Health-based failover can support resilience. Latency routing can improve user experience globally.

The AWS CLI is often the first tool that moves a learner from manual console actions to repeatable operations. Once teams become comfortable with CLI usage, they naturally progress into shell scripting, CI/CD automation, infrastructure validation, and eventually infrastructure as code.

CloudFormation matters because consistent environments are difficult to maintain manually. Template-driven provisioning reduces configuration drift and makes reviews possible. However, teams still need discipline around modular design, parameter management, and change review.

CodeCommit, CodePipeline, CodeBuild, and CodeDeploy form an AWS-native CI/CD chain. Even if a real organization uses GitHub Actions, GitLab, Jenkins, or other tooling, understanding the AWS-native equivalents improves architectural flexibility and exam readiness.

CodePipeline is the orchestrator, CodeBuild runs builds, and CodeDeploy manages rollout to targets. The deeper lesson is to separate source retrieval, build, test, approval, and deployment responsibilities clearly so pipelines are easier to reason about and troubleshoot.

Serverless and Event-Driven Deep Dive

Lambda is strongest when the problem is event-driven, bursty, or operationally simple enough that server management would be unnecessary overhead. However, Lambda is not a universal replacement for every compute model. Long-running processes, highly stateful workloads, or complex dependency-heavy applications may fit containers or virtual machines better.

EventBridge is central to loosely coupled architecture. Instead of one service

directly calling another everywhere, services emit events and rules determine what reacts. This improves composability and reduces tight coupling. When used well, EventBridge makes automation cleaner and more scalable.

CloudWatch alarms, EventBridge rules, Lambda automation, and SNS notifications together form a powerful automation pattern. For example, a state change or alarm can trigger remediation, notifications, ticket creation, or compliance actions without manual intervention.

Containers and Kubernetes Deep Dive

ECR solves the image storage side of container adoption. Teams should understand repository structure, image tagging, vulnerability scanning, and lifecycle cleanup. Container image hygiene is part of operational maturity because old or untracked images create risk and confusion.

ECS is often the simpler AWS-native choice for running containers. It integrates tightly with AWS services and can be easier for teams that do not need Kubernetes-specific capabilities. Fargate pushes simplification further by removing server management from the container host layer.

EKS is valuable when teams need Kubernetes portability, ecosystem tooling, or more complex orchestration patterns. It brings flexibility and ecosystem power, but also more operational surface area. A good architect should be able to explain why a workload needs EKS instead of defaulting to it because it sounds more advanced.

A common interview comparison is ECS versus EKS. The best answer is not that one is always better. The real decision depends on team skill set, ecosystem needs, operational tolerance, workload patterns, and platform standardization goals.

Systems Manager and Administrative Access Deep Dive

Systems Manager improves operations because it centralizes actions that would otherwise require direct host access. Session Manager is especially valuable because it lets administrators connect to instances without exposing SSH publicly. That improves auditability and reduces attack surface.

Run Command supports remote execution at scale. Patch Manager helps with update hygiene. Automation documents allow repeated operational workflows. These capabilities become very important in fleets of instances where manual access patterns stop scaling.

Terraform Deep Dive

Terraform uses declarative configuration and state to manage infrastructure. The most

important operational concept in Terraform is state management. Teams must know where state lives, who can change it, how it is locked, and how secrets inside state are protected.

Modules improve reuse, but they should stay understandable. Over-abstracted modules can make debugging harder. Good Terraform design balances reuse with clarity.

Terraform and CloudFormation both solve infrastructure as code, but their trade-offs differ. CloudFormation is deeply AWS-native. Terraform is multi-cloud and very popular in DevOps teams. The best choice depends on team preference, ecosystem, and governance model.

Migration and Production Design Deep Dive

The classic migration strategies help teams choose realistic approaches rather than forcing every system into the same modernization pattern. Some workloads can be rehosted quickly. Others deserve refactoring. Others should be retired entirely. Good migration planning is as much about business value and risk as it is about technical possibility.

A production-ready AWS application usually combines multiple concerns: DNS, ingress, network segmentation, secrets handling, compute scaling, persistence, monitoring, and deployment automation. That is why real AWS design discussions often span many services at once rather than focusing on a single tool in isolation.

AMI Deep Dive

An Amazon Machine Image is a reusable blueprint for EC2 instances. It captures the operating system, installed software, configuration baseline, and sometimes even application dependencies. The operational value is consistency: if the same AMI is used repeatedly, teams reduce drift and make server launches faster and more predictable.

In mature environments, AMIs are often baked through automated pipelines rather than created manually. This supports patching discipline, image versioning, rollback, and repeatable builds. A strong AMI process also ensures that secrets are never baked into images and that old images are retired once they are no longer safe to launch.

Scalability and High Availability Deep Dive

Scalability is about handling growth in traffic or workload demand, while high availability is about continuing service when failures happen. These two ideas work together but should not be treated as identical. A system may scale well and still be fragile, or stay online during failure but struggle to absorb increased traffic efficiently.

On AWS, scalable and highly available systems usually combine stateless application layers, load balancing, autoscaling, managed data services, and multi-AZ design. The deeper architectural lesson is that cloud systems should assume change and partial failure as normal rather than exceptional.

Auto Scaling Group Deep Dive

An Auto Scaling Group does more than launch extra servers during traffic spikes. It also acts as a self-healing mechanism by replacing unhealthy instances and maintaining the desired fleet size. This makes it a reliability feature as much as a capacity feature.

Target tracking policies are often the easiest starting point because they keep a metric such as CPU utilization near a chosen threshold. Step scaling and scheduled scaling are useful when demand patterns are more explicit. Good ASG design also depends on launch templates, health checks, cooldowns, and warm-up timing, because poor tuning can cause unnecessary churn.

Other Compute Services Deep Dive

AWS provides several compute models beyond EC2 because workloads vary in how much control, packaging, and operational overhead they require. Lambda is ideal for event-driven functions, ECS and EKS are for containers, Elastic Beanstalk simplifies application deployment, and Lightsail targets smaller simpler hosting use cases.

The real design skill is selecting the compute model that matches the workload. If the application needs full host-level control, EC2 may still be correct. If the priority is reducing server management, serverless or managed container services usually fit better.

Deployment Strategies Deep Dive

Deployment strategy determines how safely new application versions reach production. Rolling deployments update instances gradually. Blue-green deployments switch traffic between old and new environments. Canary deployments expose a small percentage of users first so behavior can be observed before a wider rollout.

On AWS, these patterns are implemented with services like CodeDeploy, load balancers, Route 53 weighted routing, or container orchestration platforms. Safe deployments depend on health checks, observability, rollback planning, and database compatibility just as much as they depend on tooling.

AWS Global Infrastructure Deep Dive

AWS global infrastructure is built from regions, Availability Zones, edge locations, and specialized points of presence. Regions support geographic separation and regulatory alignment. Availability Zones provide isolated failure domains inside a region so applications can improve fault tolerance without leaving their selected geography.

Multi-AZ architecture is common for production systems that need resilience within one region. Multi-region architecture is a bigger decision used for disaster recovery, global performance, or higher availability goals. Edge infrastructure matters for CloudFront, Route 53, and request handling close to users.

Cloud Integrations Deep Dive

Cloud integrations are what allow services to communicate cleanly without becoming tightly coupled. In AWS, that often means designing with queues, events, notifications, APIs, and workflow orchestration instead of building direct hard-coded dependencies everywhere.

SNS, SQS, EventBridge, Step Functions, and API Gateway each solve different integration problems. The most important operational concepts are retries, dead-letter queues, idempotency, and timeout handling, because integrations must continue working even when downstream systems are slow or temporarily unavailable.

Machine Learning Deep Dive

AWS machine learning services range from prebuilt AI capabilities to full custom model platforms. Rekognition, Textract, Comprehend, Polly, and Transcribe help teams add specific ML features quickly, while SageMaker supports end-to-end model development, training, tuning, and hosting.

The usual architectural question is whether the problem needs a prebuilt managed capability or a custom ML workflow. Choosing correctly reduces complexity, cost, and time to value. Like any other AWS workload, ML systems also depend on storage, identity, monitoring, and governance around the model lifecycle.

Account Management and Billing Operations Deep Dive

AWS account design affects security, cost control, and operational clarity. Separate accounts for development, staging, production, shared services, and security workloads reduce blast radius and make governance more manageable. AWS Organizations, consolidated billing, and service control policies are central to this model.

Billing operations improve significantly when teams use consistent tagging, budget alerts, and ownership boundaries. Without those, cost analysis becomes guesswork. Strong account management is one of the signs that an AWS environment is moving from

experimentation to mature operations.

Advanced Identity Deep Dive

Advanced identity on AWS focuses on federation, centralized workforce access, and temporary credentials rather than long-lived IAM users. IAM Identity Center helps organizations connect directory-based users to AWS accounts and applications in a more controlled way.

Cross-account role assumption is especially important in multi-account environments. It lets people and automation access other accounts temporarily with clear trust relationships. Permission boundaries, session policies, MFA requirements, and conditional access provide additional controls when organizations need more precise identity governance.

Other Services Deep Dive

Many AWS architectures rely on supporting services that are not always the headline topic but are still critical. SNS supports notifications and fan-out messaging. SQS provides durable decoupling. SES handles email workflows. Kinesis supports streaming ingestion. Step Functions coordinates workflows. API Gateway publishes managed APIs.

These services often sit between larger systems, which means they shape reliability, retries, visibility, and scaling patterns across the platform. Learning them by problem category rather than memorization makes them much easier to choose correctly during design discussions.

AWS Architecting and Ecosystem Deep Dive

AWS architecture is about combining services into a system that meets business needs securely, reliably, and efficiently. Strong design thinking includes identity, networking, compute, data, observability, deployment automation, and recovery planning rather than focusing on one service in isolation.

The Well-Architected mindset is useful here: operational excellence, security, reliability, performance efficiency, cost optimization, and sustainability. The broader AWS ecosystem also includes partner tools, open-source infrastructure projects, CI/CD platforms, observability tooling, and hybrid-cloud integrations that shape how AWS is used in real companies.

Interview and Exam Preparation Deep Dive

For interviews, focus on decision-making language. Instead of only defining a service, explain when to use it, why it fits, what trade-offs it brings, and which alternatives were considered.

For certification-style questions, the fastest path is often elimination. If an answer requires more operational effort than a managed AWS option with the same outcome, it is often not the best cloud-native choice. Likewise, if the question emphasizes elasticity, resilience, low operational overhead, or serverless behavior, that should immediately narrow the answer set.

The strongest preparation combines three things:

- service awareness
- architectural trade-off understanding
- hands-on familiarity with at least small labs and deployment flows